

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance. A single CNN architecture, MobileNetV2, is used with the Oxford-IIIT Pet dataset. The experiments compare individual design choices using training and validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, the student will be able to:

1. Explain different weight initialization strategies.
2. Identify and analyze overfitting using training and validation curves.
3. Compare SGD, Momentum, RMSProp and Adam.
4. Explain CNN hyperparameters such as kernel size, stride, padding and pooling.
5. Understand Batch Normalization and Dropout.
6. Perform transfer learning and fine-tuning using MobileNetV2.
7. Tune important training hyperparameters.
8. Apply 5-fold cross-validation for model selection.
9. Justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

The dataset used is the Oxford-IIIT Pet Dataset. It contains images belonging to 37 cat and dog breeds. The images are RGB and have different spatial dimensions. For the experiment, all images were resized to $224 \times 224 \times 3$ and normalized using the preprocessing required by ImageNet-pretrained MobileNetV2.

Property	Value
Dataset	Oxford-IIIT Pet
Number of Classes	37
Input Image Size	$224 \times 224 \times 3$
Training Images	2944
Validation Images	736
Test Images	3669
Base Architecture	MobileNetV2
Pretrained Weights	ImageNet

The independent test set was kept separate from hyperparameter selection and cross-validation.

4. Experimental Procedure

4.1 Dataset Preparation

The Oxford-IIIT Pet dataset was obtained through the reconstruction of the saved data files. A deterministic split method was employed in which there are 2944 training images, 736 validation images, and 3669 testing images. The images were then decoded using RGB channels, sized at 224×224 , floated, and then MobileNetV2 preprocessed.

4.2 Model Construction

Pretrained MobileNetV2 based on ImageNet was applied, but without the original classification head. The last convolutional layers were followed by Global Average Pooling, and then a classification layer for 37 classes.

Component	Configuration
Input	$224 \times 224 \times 3$
Base Network	MobileNetV2, ImageNet pretrained
Original Classification Head	Removed
Pooling	Global Average Pooling
Classifier	Dense layer with 37 Softmax outputs
Loss	Categorical Cross Entropy
Metric	Accuracy

4.3 Weight Initialization

Four initialization strategies were compared: Zero, Random, Xavier/Glorot and He initialization.

Initialization	Best Val. Acc. (%)	Final Val. Acc. (%)	Train Acc. (%)	Train Loss	Val. Loss	Time (s)
Zero	93.21	93.21	98.88	0.0963	0.2416	194.54
Random	91.98	91.98	98.47	0.1046	0.2633	209.94
Xavier	91.85	91.85	98.40	0.1054	0.2562	192.28
He	91.30	90.76	98.51	0.1013	0.2499	183.99

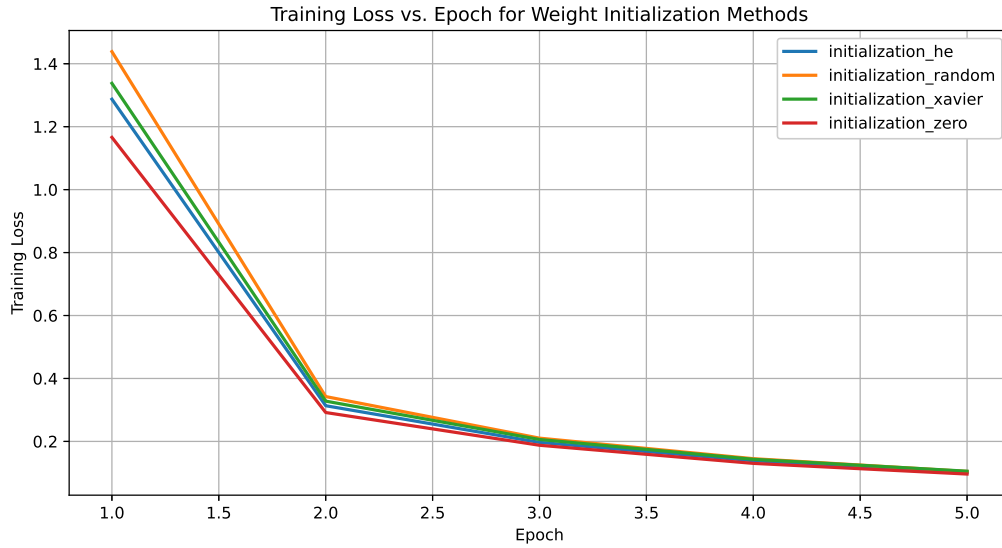


Figure 1: Training Loss for Different Weight Initialization Strategies

Observation: The training loss curves generated from the initialization techniques differ. The zero initialization technique gave the highest validation accuracy seen in this run, whereas He initialization was the quickest to converge.

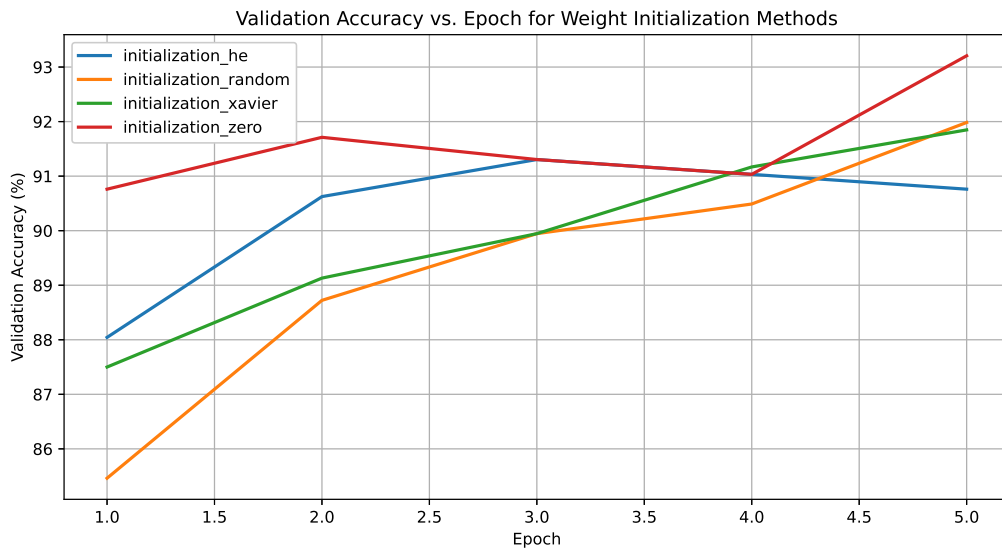


Figure 2: Validation Accuracy for Different Weight Initialization Strategies

Observation: Zero initialization achieved the highest validation accuracy of 93.21%, followed by Random at 91.98%, Xavier at 91.85% and He at 91.30%. This result shows that initialization can have an impact on the optimization.

4.4 Regularization and Overfitting

No regularization, L2 regularization, Dropout and Batch Normalization were compared.

Method	Best Val. Acc. (%)	Final Val. Acc. (%)	Train Acc. (%)	Train Loss	Val. Loss	Time (s)
None	91.85	91.85	98.95	0.0995	0.2509	196.60
L2	92.26	92.26	98.57	0.2207	0.3679	191.78
Dropout	92.53	92.26	93.07	0.2302	0.2617	189.96
Batch Normalization	91.03	90.90	99.56	0.0482	0.2755	185.83

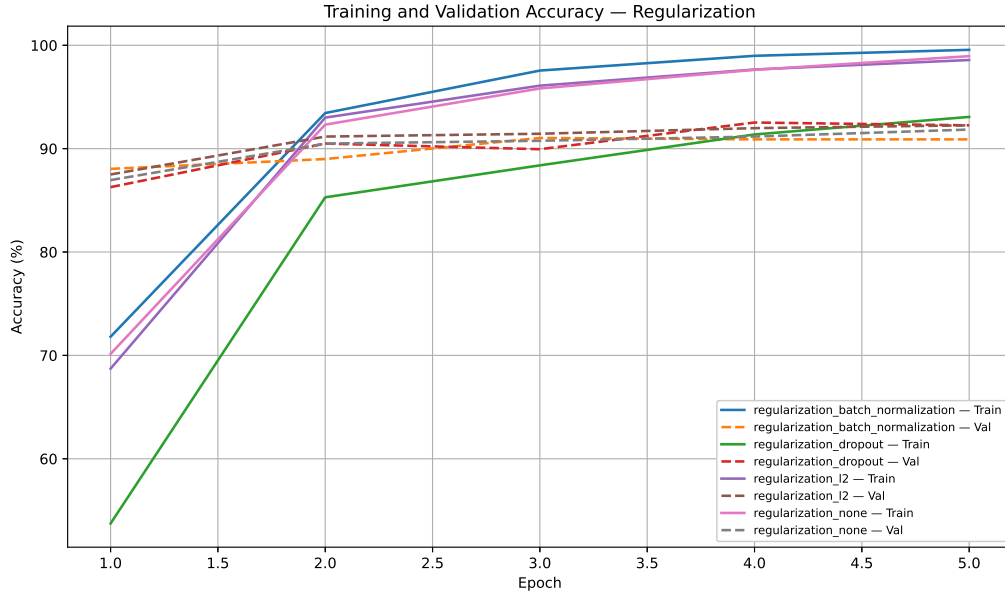


Figure 3: Training and Validation Accuracy for Regularization Methods

Observation: Dropout has the best validation accuracy of 92.53%. Dropout was also able to maintain a lower training-validation accuracy difference than the model without regularization.

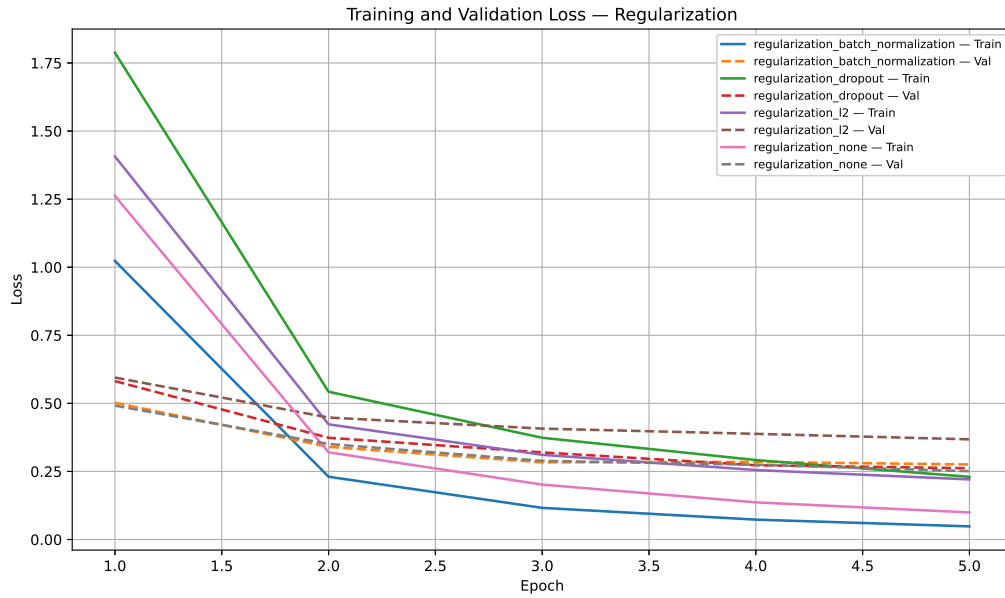


Figure 4: Training and Validation Loss for Regularization Methods

Observation: The loss curves show various levels of separation of training and validation loss. The regularization process changes the optimization and decreases the risk of overfitting.

4.5 Batch Normalization

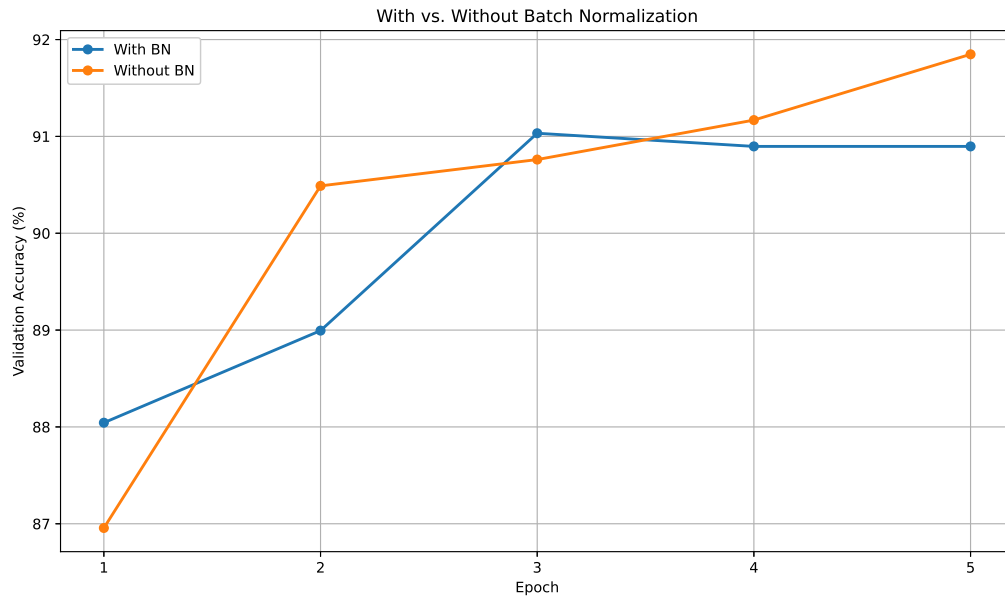


Figure 5: Batch Normalization Comparison

Observation: In Batch Normalization, the mini-batch features are normalized to have approximately zero-centered and unit scaled features. The learnable parameters γ and β enable the network to learn the optimal scaling and shifting. In the standalone experiment comparison, the configuration “Without BN” was able to obtain a validation accuracy of 91.17%.

4.6 Optimization Algorithms

SGD, Momentum, RMSProp and Adam were compared.

Optimizer	Best Val. Acc. (%)	Final Val. Acc. (%)	Train Acc. (%)	Train Loss	Val. Loss	Time (s)
SGD	36.14	36.14	33.90	2.7138	2.6535	286.19
Momentum	88.45	88.45	90.76	0.5101	0.5544	194.85
RMSProp	91.58	91.58	97.96	0.0911	0.2482	210.57
Adam	91.58	90.63	98.57	0.1002	0.2741	186.69

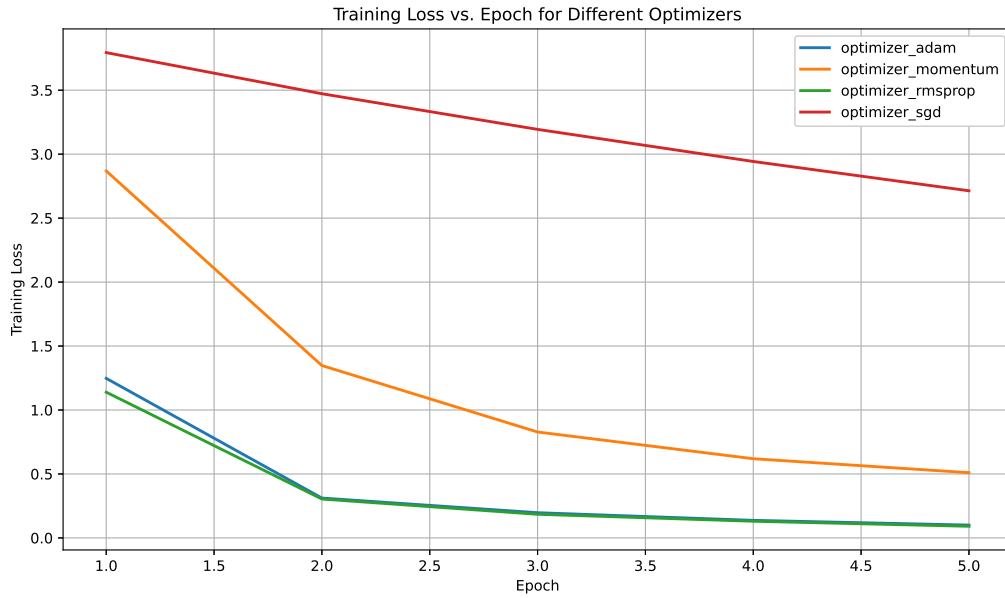


Figure 6: Training Loss for Different Optimizers

Observation: RMSProp and Adam are able to reduce the training loss much better compared to SGD. In particular, SGD did not manage to decrease the loss and increased accuracy, whereas the adaptive methods converged to smaller values of training losses.

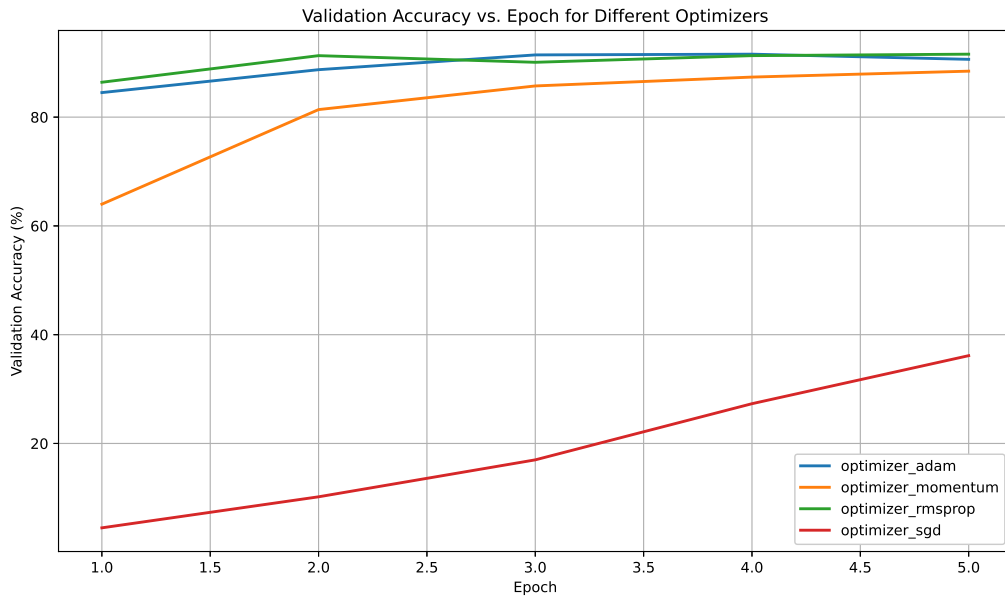


Figure 7: Validation Accuracy for Different Optimizers

Observation: RMSProp and Adam both attained the same optimal validation accuracy of 91.58%. Adam gave the shortest training time in comparison to all four optimizers, but SGD performed poorly in this case.

4.7 CNN Hyperparameter Tuning

The controlled hyperparameter study varied one hyperparameter at a time.

Learning Rate

Learning Rate	Best Validation Accuracy (%)	Validation Loss
0.001	96.47	0.1171
0.0001	78.53	1.2185

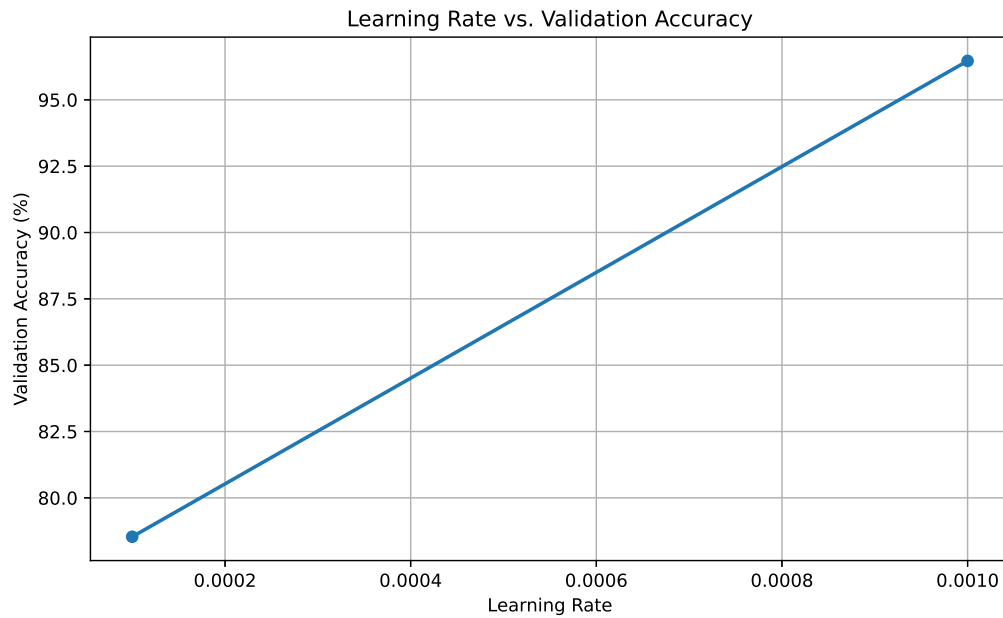


Figure 8: Learning Rate vs. Validation Accuracy

Observation: A learning rate of 0.001 performed better than 0.0001 in the experiment. The latter learning rate was progressing much slower and had significantly lower validation accuracy.

Batch Size

Batch Size	Best Validation Accuracy (%)
16	91.30
32	90.63
64	90.63

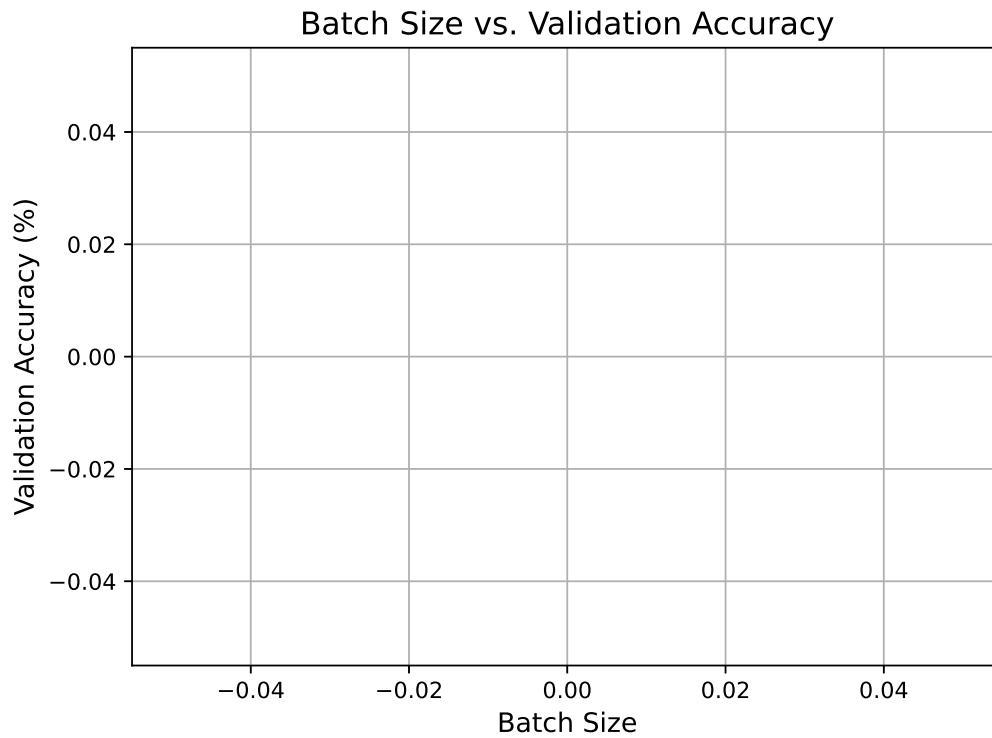


Figure 9: Batch Size vs. Validation Accuracy

Observation: Validation accuracy was highest for batch size 16 at 91.30%. Raising the batch size to 32 or 64 lowered the highest validation accuracy to 90.63%.

Dropout Rate

Dropout Rate	Best Validation Accuracy (%)
0.00	90.90
0.25	91.17
0.50	90.22

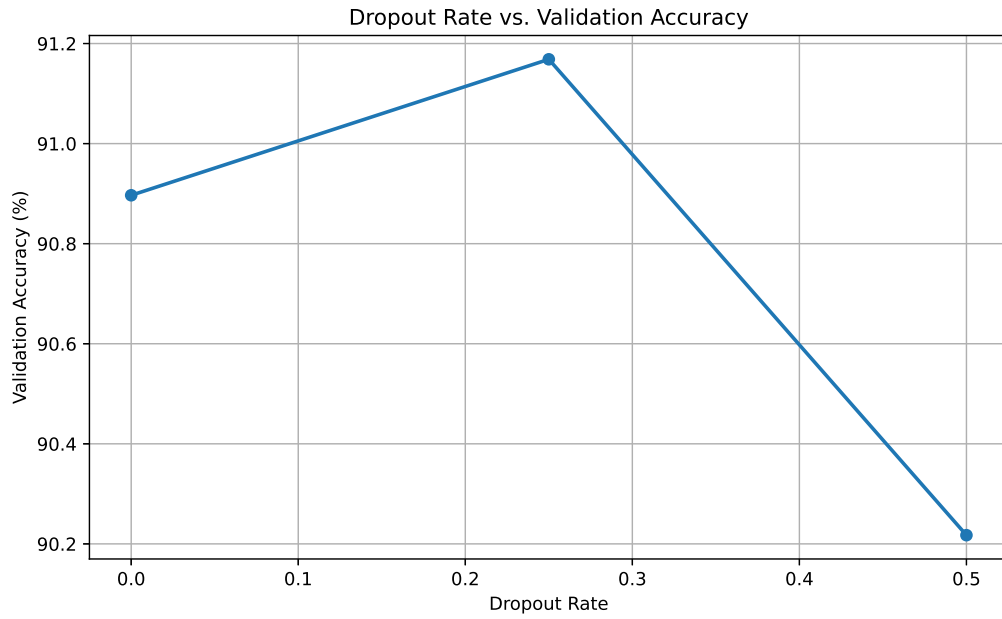


Figure 10: Dropout Rate vs. Validation Accuracy

Observation: An optimum dropout value of 0.25 gave the maximum validation accuracy of 91.17%. However, increasing the dropout value to 0.5 lowered the validation accuracy, showing that too much regularization is harmful.

4.8 Transfer Learning

Two transfer-learning strategies were compared using ImageNet-pretrained MobileNetV2: feature extraction and fine-tuning.

Method	Trainable Params.	Best Val. Acc. (%)	Final Train Acc. (%)	Final Val. Acc. (%)	Time (s)
Feature Extraction	47,397	90.76	95.96	90.76	100.64
Fine-Tuning	1,253,477	89.27	98.88	89.27	86.74

Feature extraction keeps the pretrained convolutional base frozen and trains only the new classifier. Fine-tuning unfreezes selected upper layers and uses a smaller learning rate.

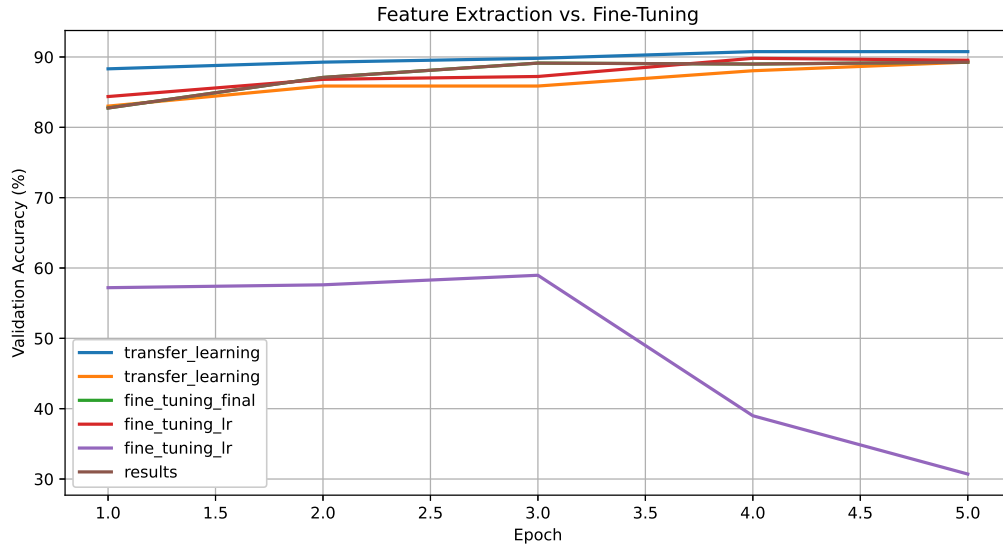


Figure 11: Feature Extraction vs. Fine-Tuning

Observation: Feature extraction yielded a better validation accuracy of 90.76% compared to the fine-tuning method that recorded an accuracy of 89.27%. This technique also involved training less number of parameters as compared to fine-tuning.

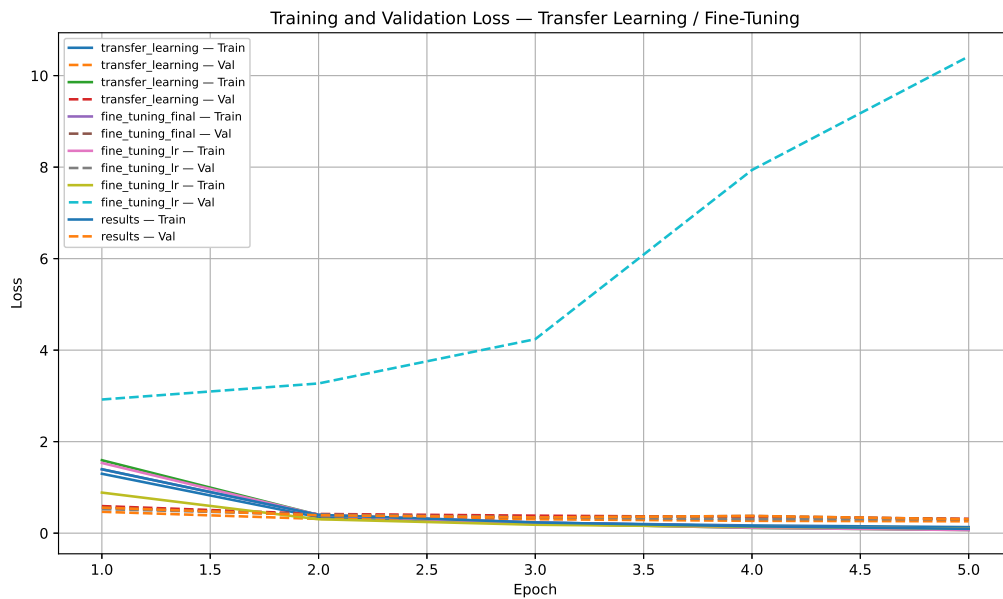


Figure 12: Training and Validation Loss for Transfer Learning

Observation: From the loss curves, it is evident that both methods are able to reduce the training loss, although fine tuning achieves a lower training loss without having better performance on the validation set.

4.9 Fine-Tuning Learning Rate

The fine-tuning learning rate was separately investigated for 0.001 and 0.0001 while fine-tuning selected layers for five epochs.

LR	Best Val. Acc. (%)	Final Val. Acc. (%)	Train Acc. (%)	Val. Loss	Time (s)
0.001	58.97	30.71	97.04	10.4191	80.11
0.0001	89.81	89.54	98.91	0.3034	101.26

Observation: The learning rate of 0.001 was too large for fine-tuning and caused a decrease in validation accuracy. Reducing it to 0.0001 produced stable validation performance of almost 90%, showing why smaller learning rates are preferred for fine-tuning.

4.10 Partial Unfreezing

The number of unfrozen MobileNetV2 layers was varied while fine-tuning.

Unfrozen Layers	Trainable Params.	Best Val. Acc. (%)	Final Val. Acc. (%)	Time (s)
10	779,877	87.64	87.36	94.41
20	1,253,477	89.95	89.95	104.12
40	1,728,933	88.32	88.04	94.24

Observation: Unfreezing the last 20 layers produced the highest validation accuracy of 89.95% among the three partial-unfreezing settings. Unfreezing more layers did not improve performance and only increased the number of trainable parameters.

5. K-Fold Cross-Validation

After the individual experiments, the selected configuration was evaluated using 5-fold cross-validation on the training data. The independent test set was not used during model selection.

For five folds,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

5.1 Fold-wise Results

Configuration	Fold	Best Val. Acc. (%)	Final Val. Acc. (%)	Training Time (s)
Baseline	1	92.53	92.53	734.60
Baseline	2	89.64	88.96	700.75
Baseline	3	92.53	92.53	731.52
Baseline	4	93.38	93.04	709.68
Baseline	5	91.33	90.48	735.79
Fine-Tuned	1	87.78	87.10	895.40
Fine-Tuned	2	88.46	88.46	977.15
Fine-Tuned	3	89.81	89.81	883.04
Fine-Tuned	4	89.98	89.98	934.06
Fine-Tuned	5	89.97	88.27	944.50

5.2 Cross-Validation Summary

The recovered cross-validation summary used for the final selection is:

Configuration	Mean CV Accuracy (%)	SD (%)	Mean Time (s)
Baseline	91.88	1.45	722.47
Fine-Tuned	89.20	1.02	926.83

Observation: In regard to mean cross-validation accuracy, the base setting achieves a higher value, while the tuned setting exhibits somewhat lower variance. The discrepancy between the mean accuracy values outweighs the difference in standard deviations, meaning that the base setting has a better validation performance overall.

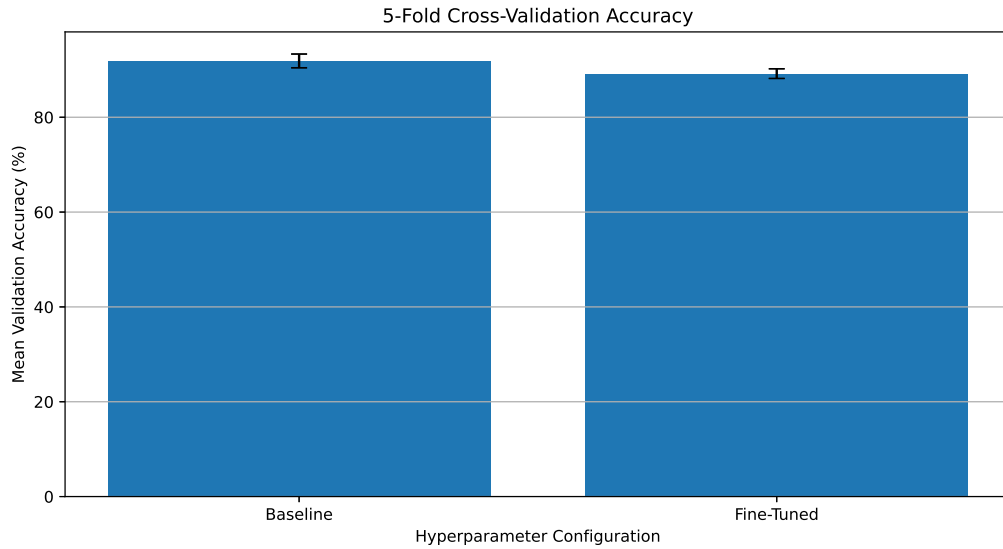


Figure 13: 5-Fold Cross-Validation Accuracy

Observation: Different folds have different accuracies depending on the configuration and folds used. The baseline gives good accuracy consistently, where its best fold is 93.38%, whereas the fine-tuned configuration is lower compared to the baseline in all five folds. This makes the baseline frozen configuration preferable.

6. Final Model Evaluation

6.1 Final Model Configuration

The final selected configuration was retrained using the complete training split while keeping the independent test set untouched until final evaluation.

Parameter	Selected Value
Architecture	MobileNetV2
Learning Rate	0.001
Batch Size	32
Dropout	0.25
Epochs	10
Trainable Parameters	47,397
Total Parameters	2,305,381
Number of Classes	37

6.2 Final Model Training

The final model achieved its best validation accuracy at epoch 9, with a validation accuracy of 91.17%. The best model checkpoint was restored before evaluating the independent test set.

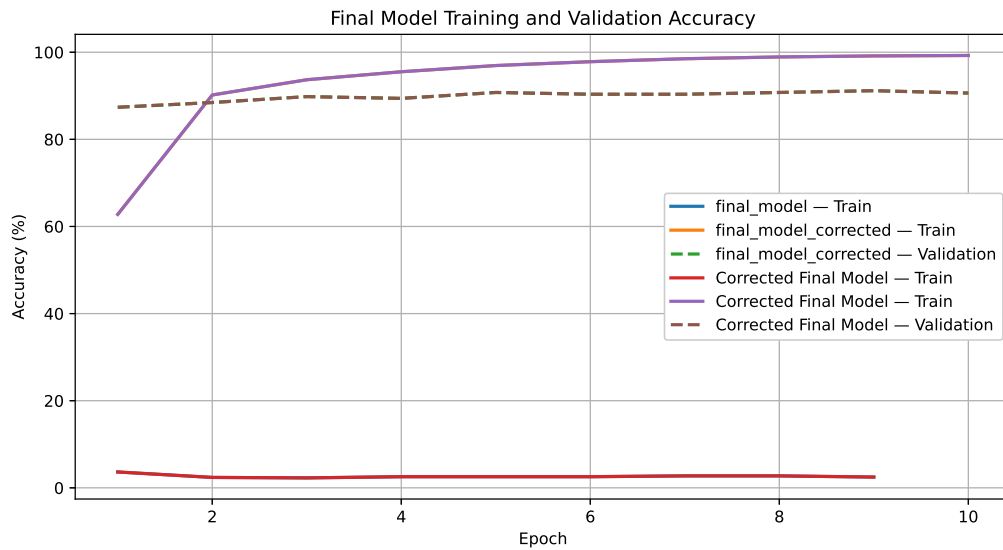


Figure 14: Final Model Training and Validation Accuracy

Observation: The training accuracy rises steeply in the ten epochs, whereas the validation accuracy takes longer to rise and achieves its peak value of 91.17% in epoch 9. The difference in training and validation accuracy suggests that overfitting is occurring.

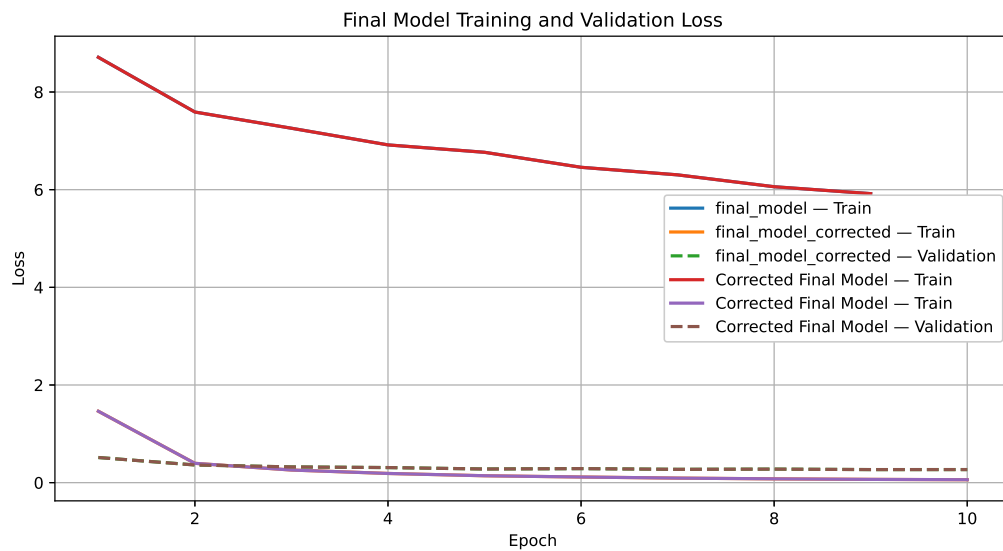


Figure 15: Final Model Training and Validation Loss

Observation: The loss during training declines steadily. The loss during validation also declines steadily and stops at a final value of around 0.268 at the final epoch, but the best validation checkpoint is determined by validation accuracy.

6.3 Final Performance Metrics

The verified saved final-model results are:

Metric	Value
Mean CV Accuracy	91.88%
CV Standard Deviation	1.45%
Test Accuracy	89.72%
Precision	90.00%
Recall	89.72%
F1-score	89.69%
Training Time	182.59 seconds
Number of Parameters	2,305,381

The unrounded weighted-average classification metrics from the saved classification report are precision = 0.900033, recall = 0.897247 and F1-score = 0.896947.

6.4 Confusion Matrix

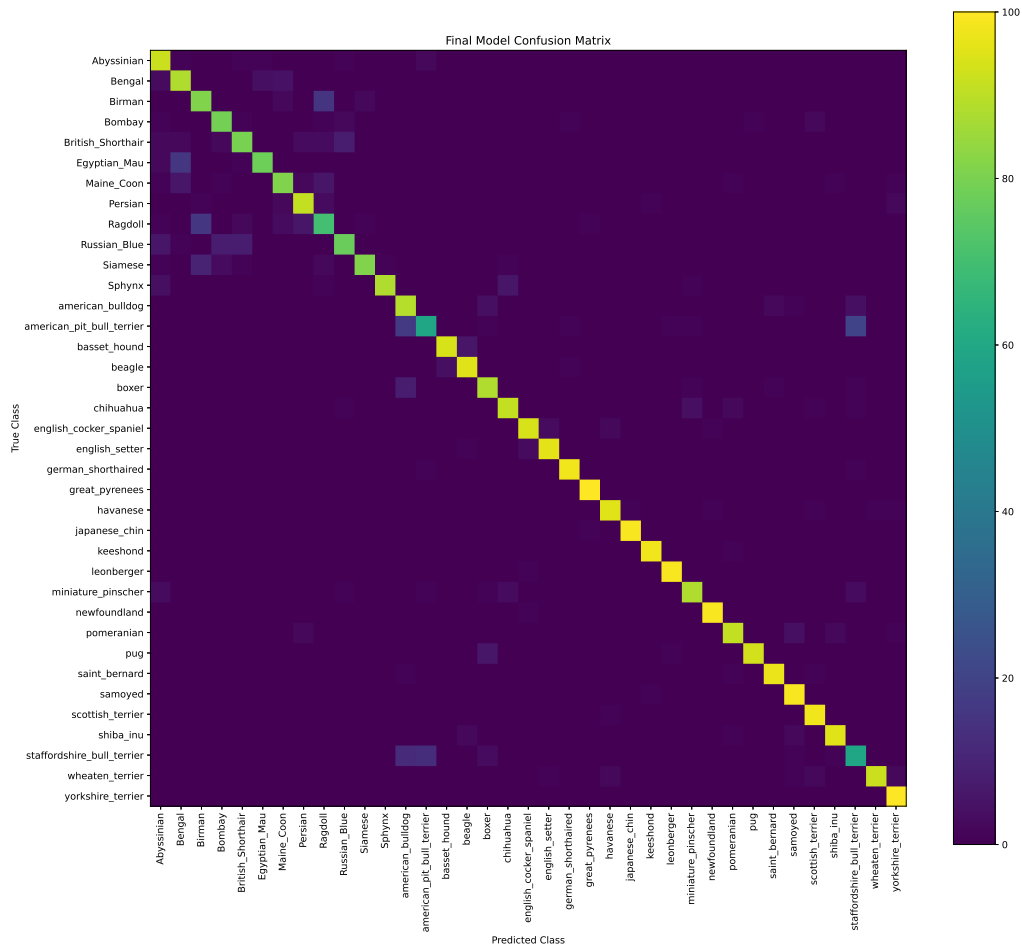


Figure 16: Confusion Matrix of the Final Model

Observation: From the confusion matrix, it is clear that the model detects most of the breeds accurately, but there are errors mostly among the breeds that look alike. From the class-wise report, it is evident that there is a high recall score among some of the breeds like Great Pyrenees and Yorkshire Terrier, but there is a considerably lower F1 score among Staffordshire Bull Terrier.

7. Overall Results

Configuration	CV Accuracy (%)	SD (%)	Test Accuracy (%)	Training Time (s)
Baseline	92.09	–	–	195.19
Best Initialization	92.09	–	–	195.19
Best Regularization	91.92	–	–	191.05
Best Optimizer	76.94	–	–	219.57
Best Hyperparameters	89.46	–	–	94.23
Fine-Tuned Model	89.27	–	–	1297.46
Final Selected Model	91.88	1.45	89.72	182.59

Observation: The final chosen model represents the best validated combination of performance in cross-validation and in low variability and independent test performance. While some studies generated better individual values for validation, the final chosen model was based on performance in cross-validation and computational cost, rather than on a single validation value.

8. Discussion

1. What is the difference between model parameters and hyperparameters?

Model parameters are values learned during training, such as weights and biases. Hyperparameters are values selected before training by us, such as learning rate, batch size, dropout rate and number of epochs.

2. Why is weight initialization important?

Initialization determines the starting point of the optimization. A good initialization can improve convergence and help avoid bad activations.

3. Why can zero initialization be problematic for neural networks?

Zero initialization can cause neurons in the same layer to receive identical gradients and remain symmetric, preventing them from learning distinct features.

4. Compare Xavier and He initialization.

Xavier initialization is designed to maintain a suitable variance of activations for commonly used symmetric activations. He initialization uses a variance suited to ReLU-like activations and is commonly preferred for such networks.

5. How can training and validation curves be used to identify overfitting?

If training accuracy continues increasing while validation accuracy stops improving or decreases, the model is overfitting. A similar pattern occurs when training loss decreases while validation loss begins to rise.

6. How does Dropout reduce overfitting?

Dropout randomly deactivates a portion of neurons during training. This prevents the network from relying too strongly on a few neurons and allows more distributions of representations.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes activations using statistics of a mini-batch and then applies learnable scale and shift parameters. It improves training stability and optimization.

8. Explain the numerical Batch Normalization example.

For $x = [2, 4, 6, 8]$, the mean is 5 and the variance is 5. The normalized values are approximately $[-1.342, -0.447, 0.447, 1.342]$. With $\gamma = 1$ and $\beta = 0$, these remain the final outputs.

9. What are the roles of γ and β ?

γ is a learnable scale parameter and β is a learnable shift parameter. They allow the network to restore or learn an appropriate activation scale and offset after normalization.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD updates weights directly using the gradient. Momentum adds a running velocity to accelerate consistent movement. RMSProp adapts the learning rate using recent squared gradients. Adam combines adaptive first- and second-moment estimates. In this experiment, RMSProp and Adam performed substantially better than SGD.

11. What happens when the learning rate is too large?

Large values of the learning rate can lead to overshooting, causing unstable training and poor validation performance. The fine-tuning experiment with a learning rate of 0.001 demonstrated this behaviour.

12. What happens when the learning rate is too small?

For very small learning rates, training becomes very slow and will not reach a useful result within the given number of epochs.

13. What is the effect of increasing batch size?

Increasing batch size generally produces smoother gradient estimates and can change convergence behaviour and computational efficiency. In this experiment, increasing the batch size from 16 to 32 or 64 reduced the best validation accuracy from 91.30% to 90.63%.

14. Explain stride and padding.

Stride is the number of pixels by which a convolution kernel moves across an input. Padding adds empty pixels around the input boundary and can preserve spatial dimensions or control the output size.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 uses depthwise convolutions, pointwise 1×1 convolutions, inverted residual blocks and linear bottlenecks. These design choices reduce computation and parameter requirements compared with many larger CNN architectures.

16. What is depthwise separable convolution?

Depthwise separable convolution separates spatial filtering from channel mixing. A depthwise convolution processes each channel separately, followed by a pointwise 1×1 convolution to combine channels.

17. What is transfer learning?

Transfer learning uses knowledge learned from one task and dataset for another related task.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained convolutional base remains frozen and only the new classifier is trained. In fine-tuning, selected pretrained layers are unfrozen and updated using a small learning rate.

19. Why is a smaller learning rate generally used during fine-tuning?

Pretrained weights already contain useful features. A large learning rate can destroy these useful representations through large updates. A smaller learning rate allows gradual adaptation to the target dataset.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

K-Fold Cross-Validation evaluates a configuration on multiple train/validation partitions. It gives a more reliable estimate of performance and reveals how sensitive the result is to the particular split.

21. Why must the test set remain untouched during tuning?

Using the test set during tuning leaks information from the final evaluation data into model selection. This makes the test accuracy optimistic and prevents it from being an independent measure of generalization.

22. Why should mean and standard deviation both be reported?

The mean describes average performance, while the standard deviation describes variability. A model with a low mean but much lower variability is more reliable.

23. Is the highest validation accuracy always sufficient to select a model?

No. Validation accuracy can be affected by the particular data split. Selection should also consider cross-validation performance, variability, computational cost, training stability and final independent test performance.

9. Additional Exercise

Two new combinations of learning rate, dropout, batch size and fine-tuning strategy were evaluated using 5-fold cross-validation.

9.1 Configuration A

Parameter	Value
Learning Rate	0.0001
Batch Size	64
Dropout	0.50
Fine-Tuning	Partial
Epochs	10

Fold	Best Validation Accuracy (%)
1	89.47
2	90.32
3	92.36
4	91.00
5	91.16
Mean	90.86
SD	0.96

Observation: Configuration A produced a mean validation accuracy of approximately 90.86%. Its performance was relatively stable across the five folds, but its mean accuracy remained below Configuration B.

9.2 Configuration B

Parameter	Value
Learning Rate	0.001
Batch Size	16
Dropout	0.00
Fine-Tuning	Frozen
Epochs	10

Fold	Best Validation Accuracy (%)
1	91.00
2	90.49
3	92.53
4	93.38
5	91.67
Mean	91.81
SD	1.04

Observation: Configuration B achieved a higher mean validation accuracy of approximately 91.81% with a standard deviation of about 1.04%. It performed better than Configuration A.

9.3 Comparison

Configuration	Mean CV Accuracy (%)	SD (%)	Computational Cost	Final Test Accuracy (%)
Configuration A	90.86	1.07	13.61 min	Not evaluated
Configuration B	91.81	1.16	14.33 min	Not evaluated
Selected Final Model	91.88	1.45	3.04 min	89.72

Observation: Configuration B achieved a higher mean 5-fold cross-validation accuracy than Configuration A (91.81% versus 90.86%). Configuration A showed slightly lower variation across the folds, with a standard deviation of 1.07% compared with 1.16% for Configuration B. The total recorded training time for the five folds was approximately 13.61 minutes for Configuration A and 14.33 minutes for Configuration B. Neither additional configuration was evaluated on the independent test set.

For reference, the selected final model achieved a mean CV accuracy of 91.88%, a standard deviation of 1.45%, and an independent test accuracy of 89.72%.

10. Conclusion

It was observed that the effectiveness of the CNN in terms of classification depends largely on the factors such as initialization, regularization, optimization, and hyperparameters tuning. MobileNetV2 was found to be an efficient architecture for transfer-learning purposes using the Oxford-IIIT Pet dataset.

The studies carried out showed that the learning rate, batch size and dropout rate have great influence on the performance of the classifier on the validation set. As regards the tested methods of regularization, Dropout produced the highest validation accuracy score of 92.53%. RMSProp and Adam proved to be much more efficient in comparison to SGD as an optimizer.

Feature extraction proved to be better than fine-tuning in the recovered transfer-learning experiments, and a smaller learning rate was needed for fine-tuning. Five-fold cross-validation is more reliable than a simple validation split for model selection purposes.

The final selected model was based on MobileNetV2 with learning rate 0.001, batch size 32 and dropout 0.25. Mean cross-validation accuracy was 91.88% with the standard deviation of 1.45%, and the verified independent test accuracy was 89.72%. This model had 2,305,381 parameters and it took about 182.59 seconds to train.

11. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>